OpenCHAI Infrastructure Control Plane (1)

Below is the overview I would use as the **Project Description / Master Plan**.

Vision

OpenCHAI is envisioned as a **next-generation Infrastructure Control Plane** for HPC and AI systems—not just a cluster deployment or automation tool.

Inspired by Kubernetes, NVIDIA Base Command Manager (BCM), OpenStack, and VMware vCenter, OpenCHAI will provide a unified platform to provision, configure, monitor, orchestrate, and manage the complete lifecycle of heterogeneous HPC and AI infrastructures.

The platform should support bare-metal provisioning, software lifecycle management, networking, storage, GPU infrastructure, Kubernetes, Slurm, monitoring, security, and future cloud integration through a resource-oriented, desired-state architecture.

Project Goals

Develop OpenCHAI as a scalable, modular, cloud-native platform that can:

- Provision bare-metal HPC and AI clusters
- Manage heterogeneous hardware
- Maintain desired and actual infrastructure state
- Reconcile configuration drift automatically
- Execute infrastructure workflows
- Support multi-tenancy and RBAC
- Scale from small clusters to national supercomputing deployments
- Provide REST APIs, CLI, and Web UI

- Serve as the single source of truth for HPC infrastructure

Core Philosophy

OpenCHAI is **not** an automation framework.

OpenCHAI is a **Control Plane**.

Instead of:

```
GUI
 ↓
Run Ansible Playbook
```

The architecture should follow:

```
GUI
 ↓
Resource API
 ↓
Desired State
 ↓
Controllers
 ↓
Workflow Engine
 ↓
Infrastructure
 ↓
Observed State
 ↓
Reconciliation
```

Automation tools such as Ansible, xCAT, Redfish, Slurm, and Kubernetes are implementation details behind the control plane.

High-Level Architecture

```
               React Web UI
                     |
               REST API / CLI
```

```
                    |
            OpenCHAI API Server
                    |
            PostgreSQL Database
            (Source of Truth)
                    |
          Desired State Resources
                    |
          Reconciliation Controllers
                    |
        Infrastructure Adapters
──────────────────────────────────────
 xCAT
 Slurm
 Kubernetes
 Redfish
 LDAP
 Docker
 Monitoring
 Storage
 Networking
──────────────────────────────────────
                    |
           HPC / AI Infrastructure
```

(OR)

```
React UI

CLI

REST API

API Server

Authentication

Authorization

Validation

_____________________________________
```

```
Resource Store

(PostgreSQL)

_____________________________________

Controller Manager

_____________________________________

Workflow Engine

_____________________________________

Event Bus

_____________________________________

Infrastructure Adapters

xCAT
Slurm
Kubernetes
Redfish
LDAP
Storage
Network

_____________________________________

Physical Infrastructure
```

Architectural Principles

Every design decision should follow these principles:

- Resource-oriented architecture
- Desired state management
- Continuous reconciliation
- Event-driven workflows
- API-first development
- Modular controllers
- Separation of control plane and automation
- Idempotent operations
- Scalability
- High availability
- Extensibility through adapters

Resource Model

Everything in OpenCHAI should be represented as a resource.
Resource Definition Framework.

Examples:

```
Organization

Project

Cluster

Rack

Node

Network
```

```
Subnet

Switch

GPU

Storage

Image

OSProfile

SoftwareStack

ContainerRuntime

Scheduler

User

Role

Workflow

Task

Policy

Secret

Credential

Certificate

Backup
```

```
Audit

Alert

Event
```

Each resource follows a common schema:

```
apiVersion:
kind:
metadata:
spec:
status:
conditions:
events:
history:
```

Technology Stack

Frontend

- React
- TypeScript
- Tailwind CSS
- Vite

Backend

- Python
- FastAPI

- SQLAlchemy
- Pydantic
- Alembic

Database

- PostgreSQL

Future Components

- Redis
- NATS or RabbitMQ
- Prometheus
- Grafana
- Loki
- Vault
- Keycloak

Development Roadmap

Phase 1 — Resource Model

Design:

- Resources
- Database schema
- REST APIs
- Relationships
- Folder structure

Phase 2 — Desired State Engine

Implement:

- Desired state
- Actual state
- Resource versioning
- State comparison

Phase 3 — Controller Framework

Develop controllers for:

- Provisioning
- Network
- GPU
- Storage
- Kubernetes
- Slurm
- Monitoring

Phase 4 — Workflow Engine

Implement:

- Long-running jobs
- Retry
- Rollback
- Event processing

Phase 5 — Infrastructure Adapters

Integrate:

- xCAT

- Redfish
- Slurm
- Kubernetes
- LDAP
- Docker

Phase 6 — Multi-Tenancy & Security

Develop:

- Organizations
- Projects
- RBAC
- Authentication
- Authorization
- Audit

Phase 7 — High Availability & Scale

Implement:

- Controller HA
- Database HA
- Distributed workers
- Event bus
- Horizontal scaling

Engineering Domains

The project spans multiple engineering disciplines:

- Infrastructure Control Plane Engineering

- HPC Infrastructure Engineering
- Backend Engineering
- Kubernetes Platform Engineering
- Workflow & Automation Engineering
- Frontend Engineering
- Database Engineering
- Security Engineering
- Networking Engineering
- Storage Engineering
- Platform/SRE Engineering
- AI Infrastructure Engineering

How I suggest we work

Rather than jumping into coding, I'd like us to work like an engineering organization.

Phase 0 — Architecture (mandatory)

- Vision
- Terminology
- Resource model
- Domain model
- System context
- Component architecture
- API conventions
- Database strategy
- Controller framework

- Event model

Phase 1 — Platform Foundation

- Repository structure
- Backend
- Frontend
- Authentication
- PostgreSQL
- Migration framework
- Configuration
- Logging
- Metrics

Phase 2 — Core Control Plane

- Resource engine
- Desired state
- Status reconciliation
- Controller manager
- Event bus
- Workflow engine

Phase 3 — Infrastructure

- Bare-metal provisioning
- Networking
- Storage
- GPU lifecycle
- Slurm
- Kubernetes

- Monitoring

Phase 4 — Enterprise Features

- Multi-tenancy
- RBAC
- Audit
- Policies
- HA
- Backup
- Disaster recovery

Phase 5 — Ecosystem

- SDK
- Plugins
- Adapters
- Public APIs
- Documentation
- CLI
- Marketplace (future)